

Software engineering project: analysis report

NEO-Watch



Monitoring near-Earth objects catalogued by NASA

Students (Group 7):
Valentin Denni
Yalalt Schneider
Sofia Bounaga
Mohammed-adam Lakhrissi

Advisor:
Olivier Ricciardi

2026–2027

Contents

1 Introduction **2**

2 Requirements specification **2**
2.1 Application scope . . . 2
2.2 Questions and choices . . . 2
2.2.1 Choice of the API entry point . . . 3
2.2.2 API limits and import cost . . . 3
2.2.3 Selection of the data used . . . 3
2.2.4 Import window . . . 3
2.2.5 Data updates . . . 4
2.2.6 Export formats . . . 4
2.2.7 NEOs creation by users . . . 4
2.2.8 Alerts . . . 4
2.2.9 Account . . . 5
2.2.10 Optional features . . . 5
2.3 Feature description . . . 5
2.4 Questions and choices . . . 6
2.4.1 Choice of the API entry point . . . 6
2.4.2 API limits and import cost . . . 7
2.4.3 Selection of the data used . . . 7
2.4.4 Import window . . . 7
2.4.5 Data updates . . . 8
2.4.6 Export formats . . . 8
2.4.7 NEOs creation by users . . . 8
2.4.8 Alerts . . . 8
2.4.9 Account . . . 8
2.4.10 Optional features . . . 9
2.5 Requirements and constraints . . . 9
2.6 Team work and organization . . . 9

3 General design **11**
3.1 Use case diagram . . . 11
3.2 Layered architecture . . . 12
3.3 Class diagram . . . 12
3.4 Physical data model . . . 13
3.4.1 Variable dictionary . . . 14
3.5 Packages diagram . . . 16
3.6 Sequence diagram . . . 17
3.7 Endpoint table . . . 18
3.8 Streamlit screens . . . 18

4 Components to implement and estimated durations **20**

5 Conclusion **21**

1 Introduction

Whether out of scientific interest, curiosity, or to raise awareness of space-related risks, the observation of near-Earth objects is now a topic of significant interest. Near-Earth Objects (NEOs) refer to asteroids and comets whose orbits can bring them close to Earth. Because of the potential consequences of an impact by such an object, their detection and tracking are subject to constant monitoring, particularly by NASA. NASA makes a large amount of data on NEOs available to the public, accessible notably through its API.

However, despite the richness of this data, using it directly is not necessarily tailored to a user's needs. Quickly identifying near-Earth objects, searching for a specific NEO, tracking its trajectory over time, or receiving alerts about events of potential interest requires tools that can process and present this information in a more accessible way. Access to the data alone is therefore not enough. It is necessary to provide an interface that allows users to view, organize, and customize the data according to their individual needs. NEO-Watch is part of this initiative, an app designed to facilitate access to and tracking of near-Earth objects using data provided by NASA.

This project therefore prompts us to consider the design of an application capable of leveraging external data while meeting specific needs in terms of search, customization, and tracking. This analysis report first presents the project's specifications, as well as its scope, features, requirements, and the choices made. The second part is devoted to the team's organization and work planning. The third part presents the application's overall design, including various diagrams, the chosen architecture, the data model, and the main technical components. Finally, the last part details the components to be implemented and the estimated timeframes for their completion, before concluding with a summary of the project.

2 Requirements specification

2.1 Application scope

The NEO-Watch project involves developing an application that allows users to analyze data on near-Earth objects provided by NASA. The application is designed to let users view available information on NEOs, search for specific objects, and track them more easily using customized features.

To meet these objectives, NEO-Watch includes a user account management system and a personal dashboard where each user can access their own information. The application also offers a search function for NEOs in its database, as well as the ability to create and edit a favorites list. Users can also add specific NEOs to the database and set up alerts based on the information that interests them.

The scope of the project is thus defined around three main areas: access to NEO data, searching for NEOs, and customization for each user. The various features designed to meet these objectives will be detailed in the following sections of the specifications.

2.2 Questions and choices

This section presents the choices made about the data source, the imported data and the update policy. The way the data is split into tables, the identification of objects, the protection of user-created NEOs and the storage of statistics follow from these choices and are detailed in Section 3.4.

2.2.1 Choice of the API entry point

The NASA NeoWs API offers three entry points: `/neo/browse`, which lists all the NEOs; `/feed`, which gives all the close approaches within a period; and `/neo/{id}`, which shows all the details about an object. One page of `/neo/browse` shows at most 20 NEOs. As more than 60,000 NEOs are registered, browsing the catalog requires 3,114 calls, and since each object has an average of 29 approaches, this represents about 1.8 million rows. Moreover, `/neo/browse` is organized by NEO, whereas F2, F3 and F6 work by period : each such request would require reading the whole catalog to extract a few approaches.

We therefore decided not to use `/neo/browse` and to keep only `/feed` as the source of our database. It is completed with `/neo/{id}` when a user marks a NEO as a favorite. The consequence of this choice is that we will never have access to the full catalog : a NEO that is far from Earth during our import will not be present in our database. The application focuses only on close objects, not on small objects or those too far away.

2.2.2 API limits and import cost

Filling the database has limits and a cost. With one key, we can make 2,000 requests per hour. In addition, `/feed` limits the period of a request to a maximum of seven days; beyond one week, a 400 error is raised. Filling a three-year database requires about 157 calls (1,096 days divided by 7), whereas `/neo/browse` would have required 3,114 calls. We also decided that each group member will use their own key rather than a single shared key : running many imports during development will probably require more requests than one key allows, and would therefore block the other members.

2.2.3 Selection of the data used

The API does not provide a precise diameter value, only a size interval. We could have kept the mean of the minimum and maximum of the interval, but since this number is not present in the data, we chose to keep only the minimum value. This choice is more cautious, especially for F6: with an alert such as “more than 100 m wide”, we are sure that the NEO’s diameter is effectively greater than 100 m. The complete interval is shown on the NEO’s detail page, and the minimum value is used only for sorting and filtering. For the same reasons of consistency, measurements are expressed in kilometers (km) and kilometers per hour (km/h).

NASA gives each approach distance in kilometers, miles, lunar distances and astronomical units. The lunar distance is the most understandable for an inexperienced user, but the people we aim to reach are already passionate about astronomy. Therefore, besides distances in kilometers, users can also read them in astronomical units, which we do not store since they are obtained by a simple division.

Finally, the number of potentially hazardous NEOs requested by F2 requires no computation, as the API already provides the attribute `is_potentially_hazardous_asteroid`.

2.2.4 Import window

F2 asks for upcoming approaches, which pulls towards the future, and FO3 asks for the evolution of the number of approaches, which pulls towards the past. For F5, which requires the distance history of a favorite, `/neo/{id}` returns all the approaches of the NEO in a single call, without anything having to be imported. We import two years of past and one year of future data. The past is imported once and considered frozen, with no reason to be refreshed.

Since an object passes again on average every ten years, each NEO appears only once in our window. A final restriction concerns the nature of possible statistics. Any statistic about the evolution of a characteristic over time is out of reach, and this is not a limitation of the import window but of the model itself. What FO3 can offer is of another kind : distributions over

the population of objects in the database (size distribution, proportion of potentially hazardous objects).

2.2.5 Data updates

Each daily update adds one day at the end of the one-year future window (one call) and refreshes the next thirty days with the NEOs newly discovered over that period (five calls in seven-day periods), which amounts to about six calls per day. We do not purge: the volume involved is small and there is no maintenance cost. A purge would also raise an integrity issue, since a NEO referenced by a favorite or an alert cannot disappear.

F1 gives the administrator the responsibility of triggering updates, and FO7 proposes automatic triggering. We implement both: a daily batch process and a button in the administrator space that call the same service. An ongoing import is signaled by a status in the `import` table.

2.2.6 Export formats

F3 requires an export of search results and F5 an export of distance history. Tabular data is exported as CSV. These exports trigger no call to NASA : a favorite's history was retrieved when it was marked as a favorite and lives in the database.

2.2.7 NEOs creation by users

One of the crucial points about being able to create NEOs, is that many users could create a NEO according to one recently observed. But we do not want a lot of creation just for one real NEO. In order to fix this, we chose that, to create a NEO a request has to be sent, and then an admin will accept it or not. By this process we also avoid some creations that could be made just as a joke for example. Another point is how do we know if a NEO has been created by a user or not. In the class Neo, an attribute custom is present. It is used to know if a user or the NASA has referenced the object. But as the class Neo is linked with the class Proposal, we can also know who created the NEO if necessary.

This Proposal class is needed, when the user wants to add a NEO, he can propose his idea and he gives as much information as he can, after one admin will inspect it, and then accept or decline the proposal.

2.2.8 Alerts

The goal of this function is to send a notification according to the criteria the user has chosen. In order to do it, we need a criterion that will send the alert. According to the information available from the NASA API, we will have three criteria. The first one is if a NEO with a bigger diameter than a chosen value is detected, then if the NEO is closer than a maximum distance from an object like the Earth or Venus. Finally if a precise NEO, chosen by the user is detected. All of these can be combined together. As the notification has to be sent when the user logs in, we can evaluate these criteria at this moment. Another option would have been to send the notification each time the database is updated, however it means that the user could be notified several times in one use which can be annoying for him if he needs to regularly update the database. There could be the same issue if this person logs in many times in a day but we can assume that usually people will use the application once or twice a day. Moreover, one major problem about notifications, is that we do not want to send the same notification over and over. Once it has been sent, it has to be the first and last time. In order to do that, we could have saved all the notifications sent but with time, the table could become too voluminous. One way to answer this issue, is to save only the last criteria asked by the user, and delete it once it has been shown.

2.2.9 Account

As each admin has to be promoted by another admin, we obviously need to create by ourselves the first admin. We can create a special id in the code, which has by default the admin status. And then, when a new admin is needed, the first one can upgrade an account to "admin". When the new role is accepted, nothing changes except his permissions. An admin is also a user. He can do whatever a user can do, just he has the status of "admin" which provides him other functions to use.

Once an account has been deleted, his favourites and notifications are also deleted. However, the NEOs created by the user are kept in the table. As we assume that our user will be well informed, we suppose that their creations are useful and that is the reason why we want them to be still there.

2.2.10 Optional features

We chose to add three optional features to our app. The first one is FO3. It consists of providing some other statistics to the user. This one is important for us as we suppose that our users will be passionate about astronomy, and therefore, they will request more information about the NEOs. Then we will add FO7. The principle is to update our database regularly. As this function already exists, but it has to be used manually, we want it to be automatic in case an admin cannot update it often enough.

Finally, our last choice was to choose FO8 which shows the picture of the day provided by NASA whenever the user wants. In fact, it does not provide any information, but it can be pretty pleasant for the user to see different pictures of our space as our users are probably here to inform themselves, seeing a new picture every day can push them to search about what they are looking for.

2.3 Feature description

The first six features detailed in this section correspond to the core requirements (F1 to F6) explicitly mandated by the project specifications.

The first functionality of NEO-Watch concerns user management and access to the application. The application will provide a secured access system based on user accounts. A regular user will be able to create an account and log in to access the application's features and their personal space. Administrators will have additional permissions, allowing them to manage user accounts, consult connection history and trigger a data update. An administrator will also have access to the regular features of the application, while benefiting from these additional management capabilities.

Once connected, the user will have access to a personalized dashboard providing an overview of the most relevant information available in NEO-Watch. It will notably present upcoming close approaches, potentially hazardous NEOs among these approaches, and the object expected to come closest to Earth over a given period. The dashboard will also take the user's preferences into account by displaying information related to their favourite NEOs. The objective is to provide the user with a quick overview of the current situation without requiring them to perform a search each time they use the application.

The application will also provide a browsing and search system allowing users to explore the NEOs available in the database. Several search criteria will be available, such as the name or identifier of an object, its dimensions or the date of its next close approach. The results will be sortable according to different criteria in order to make it easier to find the most relevant objects. By selecting a NEO, the user will be able to access a more detailed view containing its relevant characteristics and information about its close approaches to Earth. Search results will

also be exportable, allowing users to keep or further process the information obtained from the application.

NEO-Watch will additionally allow users to create NEOs in the application's database. This feature is intended to give users the possibility of adding an object they have observed when it is not already available in the data used by the application. These user-created objects will be kept separately from the data provided by NASA so that they remain available when the database is subsequently updated. This functionality therefore allows the database to contain both information originating from NASA and objects added directly by NEO-Watch users.

Another important feature of the application will be the management of a favourite NEO list. Users will be able to add or remove objects from their personal list and consult it at any time. The application will keep historical information about the distance of these favourite objects from Earth, making it possible to monitor their evolution over time. This history will also be used to produce graphs that can be exported by the user, providing a more visual way of following the objects they are interested in.

Finally, users will be able to create personalized alerts based on information concerning NEOs and their close approaches. For example, a user could request to be notified when an asteroid exceeding a given size passes within a specified distance of Earth. The application will regularly check the available data against the criteria defined by the user and store the corresponding notifications. These notifications will then be displayed when the user logs in, allowing them to quickly identify events matching their interests.

There will also be a feature that would provide users with additional information about the NEOs contained in the database, such as the distribution of their sizes, the proportion of potentially hazardous objects or the number of close approaches over a given period. The statistics would be presented in a way that makes them easy to interpret and relevant to the information available in NEO-Watch.

Moreover, NEO-Watch's database will be automatically reloaded. NEO-Watch will already provide administrators with the possibility of manually updating the application's data. The optional functionality would extend this mechanism by allowing the update process to be performed automatically on a regular basis. This would reduce the need for an administrator to manually maintain the database and would help keep the information used by the application up to date.

Finally, the app will give "the NASA Astronomy Picture of the Day" to the user. Users will be able to access the picture provided by NASA through its dedicated API. Although this feature is not directly related to the monitoring of NEOs, it provides a more general and engaging astronomical experience. It also gives users the opportunity to discover another aspect of NASA's available data alongside the information provided by NEO-Watch.

2.4 Questions and choices

This section presents the choices made about the data source, the imported data and the update policy. The way the data is split into tables, the identification of objects, the protection of user-created NEOs and the storage of statistics follow from these choices and are detailed in Section 3.4.

2.4.1 Choice of the API entry point

The NASA NeoWs API offers three entry points: `/neo/browse`, which lists all the NEOs; `/feed`, which gives all the close approaches within a period; and `/neo/{id}`, which shows all the details about an object. One page of `/neo/browse` shows at most 20 NEOs. As more than 60,000 NEOs

are registered, browsing the catalog requires 3,114 calls, and since each object has an average of 29 approaches, this represents about 1.8 million rows. Moreover, `/neo/browse` is organized by NEO, whereas F2, F3 and F6 work by period : each such request would require reading the whole catalog to extract a few approaches.

We therefore decided not to use `/neo/browse` and to keep only `/feed` as the source of our database. It is completed with `/neo/{id}` when a user marks a NEO as a favorite. The consequence of this choice is that we will never have access to the full catalog : a NEO that is far from Earth during our import will not be present in our database. The application focuses only on close objects, not on small objects or those too far away.

2.4.2 API limits and import cost

Filling the database has limits and a cost. With one key, we can make 2,000 requests per hour. In addition, `/feed` limits the period of a request to a maximum of seven days; beyond one week, a 400 error is raised. Filling a three-year database requires about 157 calls (1,096 days divided by 7), whereas `/neo/browse` would have required 3,114 calls. We also decided that each group member will use their own key rather than a single shared key : running many imports during development will probably require more requests than one key allows, and would therefore block the other members.

2.4.3 Selection of the data used

The API does not provide a precise diameter value, only a size interval. We could have kept the mean of the minimum and maximum of the interval, but since this number is not present in the data, we chose to keep only the minimum value. This choice is more cautious, especially for F6: with an alert such as “more than 100 m wide”, we are sure that the NEO’s diameter is effectively greater than 100 m. The complete interval is shown on the NEO’s detail page, and the minimum value is used only for sorting and filtering. For the same reasons of consistency, measurements are expressed in kilometers (km) and kilometers per hour (km/h).

NASA gives each approach distance in kilometers, miles, lunar distances and astronomical units. The lunar distance is the most understandable for an inexperienced user, but the people we aim to reach are already passionate about astronomy. Therefore, besides distances in kilometers, users can also read them in astronomical units, which we do not store since they are obtained by a simple division.

Finally, the number of potentially hazardous NEOs requested by F2 requires no computation, as the API already provides the attribute `is_potentially_hazardous_asteroid`.

2.4.4 Import window

F2 asks for upcoming approaches, which pulls towards the future, and FO3 asks for the evolution of the number of approaches, which pulls towards the past. For F5, which requires the distance history of a favorite, `/neo/{id}` returns all the approaches of the NEO in a single call, without anything having to be imported. We import two years of past and one year of future data. The past is imported once and considered frozen, with no reason to be refreshed.

Since an object passes again on average every ten years, each NEO appears only once in our window. A final restriction concerns the nature of possible statistics. Any statistic about the evolution of a characteristic over time is out of reach, and this is not a limitation of the import window but of the model itself. What FO3 can offer is of another kind : distributions over the population of objects in the database (size distribution, proportion of potentially hazardous objects).

2.4.5 Data updates

Each daily update adds one day at the end of the one-year future window (one call) and refreshes the next thirty days with the NEOs newly discovered over that period (five calls in seven-day periods), which amounts to about six calls per day. We do not purge: the volume involved is small and there is no maintenance cost. A purge would also raise an integrity issue, since a NEO referenced by a favorite or an alert cannot disappear.

F1 gives the administrator the responsibility of triggering updates, and FO7 proposes automatic triggering. We implement both: a daily batch process and a button in the administrator space that call the same service. An ongoing import is signaled by a status in the `import` table.

2.4.6 Export formats

F3 requires an export of search results and F5 an export of distance history. Tabular data is exported as CSV. These exports trigger no call to NASA : a favorite's history was retrieved when it was marked as a favorite and lives in the database.

2.4.7 NEOs creation by users

One of the crucial points about being able to create NEOs, is that many users could create a NEO according to one recently observed. But we do not want a lot of creation just for one real NEO. In order to fix this, we chose that, to create a NEO a request has to be sent, and then an admin will accept it or not. By this process we also avoid some creations that could be made just as a joke for example. Another point is how do we know if a NEO has been created by a user or not. In the class Neo, an attribute custom is present. It is used to know if a user or the NASA has referenced the object. But as the class Neo is linked with the class Proposal, we can also know who created the NEO if necessary.

This Proposal class is needed, when the user wants to add a NEO, he can propose his idea and he gives as much information as he can, after one admin will inspect it, and then accept or decline the proposal.

2.4.8 Alerts

The goal of this function is to send a notification according to the criteria the user has chosen. In order to do it, we need a criterion that will send the alert. According to the information available from the NASA API, we will have three criteria. The first one is if a NEO with a bigger diameter than a chosen value is detected, then if the NEO is closer than a maximum distance from an object like the Earth or Venus. Finally if a precise NEO, chosen by the user is detected. All of these can be combined together. As the notification has to be sent when the user logs in, we can evaluate these criteria at this moment. Another option would have been to send the notification each time the database is updated, however it means that the user could be notified several times in one use which can be annoying for him if he needs to regularly update the database. There could be the same issue if this person logs in many times in a day but we can assume that usually people will use the application once or twice a day. Moreover, one major problem about notifications, is that we do not want to send the same notification over and over. Once it has been sent, it has to be the first and last time. In order to do that, we could have saved all the notifications sent but with time, the table could become too voluminous. One way to answer this issue, is to save only the last criteria asked by the user, and delete it once it has been shown.

2.4.9 Account

As each admin has to be promoted by another admin, we obviously need to create by ourselves the first admin. We can create a special id in the code, which has by default the admin status.

And then, when a new admin is needed, the first one can upgrade an account to "admin". When the new role is accepted, nothing changes except his permissions. An admin is also a user. He can do whatever a user can do, just he has the status of "admin" which provides him other functions to use.

Once an account has been deleted, his favourites and notifications are also deleted. However, the NEOs created by the user are kept in the table. As we assume that our user will be well informed, we suppose that their creations are useful and that is the reason why we want them to be still there.

2.4.10 Optional features

We chose to add three optional features to our app. The first one is FO3. It consists of providing some other statistics to the user. This one is important for us as we suppose that our users will be passionate about astronomy, and therefore, they will request more information about the NEOs. Then we will add FO7. The principle is to update our database regularly. As this function already exists, but it has to be used manually, we want it to be automatic in case an admin cannot update it often enough.

Finally, our last choice was to choose FO8 which shows the picture of the day provided by NASA whenever the user wants. In fact, it does not provide any information, but it can be pretty pleasant for the user to see different pictures of our space as our users are probably here to inform themselves, seeing a new picture every day can push them to search about what they are looking for.

2.5 Requirements and constraints

Technical

- Python 3.13, layered architecture imposed by the template (DAO / service / controller / view)
- Imposed stack: FastAPI (backend), Streamlit (frontend), PostgreSQL (database)
- Development environment: SSP Cloud / Onyxia, no local development
- Version control and collaboration via Git

Subject-specific

- NASA API key quota (limited number of requests, to be anticipated especially for automatic updates)
- Preservation of user-created NEOs during automatic database updates
- Read-only access to the NASA API: no modification or write access on NASA's side, only NEO-Watch's own data (accounts, added NEOs, alerts, favourites) can be modified
- Data freshness: alerts and the dashboard depend on regular updates, which constrains how often synchronization with the API must occur

2.6 Team work and organization

The whole writing process of our intermediate report was based on a clear set of rules. Firstly, division of labor: everyone in the group contributed, with some members contributing more to certain sections based on differences in skill or familiarity with the topic. Every section not only

had a writer but also a designated reviewer, ensuring that each part was checked by someone other than its writer, reducing the risk of errors or inconsistencies going unnoticed.

Beyond the writing itself, the group follows two rules meant to keep both the report and the technical work consistent from the start. The first concerns sequencing: section 2.2 ("Questions and Choices") is written before any other technical section, since it requires obtaining a NASA API key and actually querying the API to understand its real data structure, request quotas, and any missing or unexpected fields. Every other technical section, from the class diagram to the endpoint table, depends on this section, so starting elsewhere would risk having to rework and correct sections a second time.

The second rule concerns vocabulary. The team shares a reference document where naming decisions are fixed once and reused consistently throughout the project, for instance, choosing between "NEO" and "Asteroid," or "Approach" and "CloseApproach." These names must remain identical across the class diagram, the physical data model, and the API endpoints; inconsistent naming between different parts is one of the most visible issues an evaluator can spot, so agreeing on terminology in advance avoids this problem entirely.

3 General design

3.1 Use case diagram

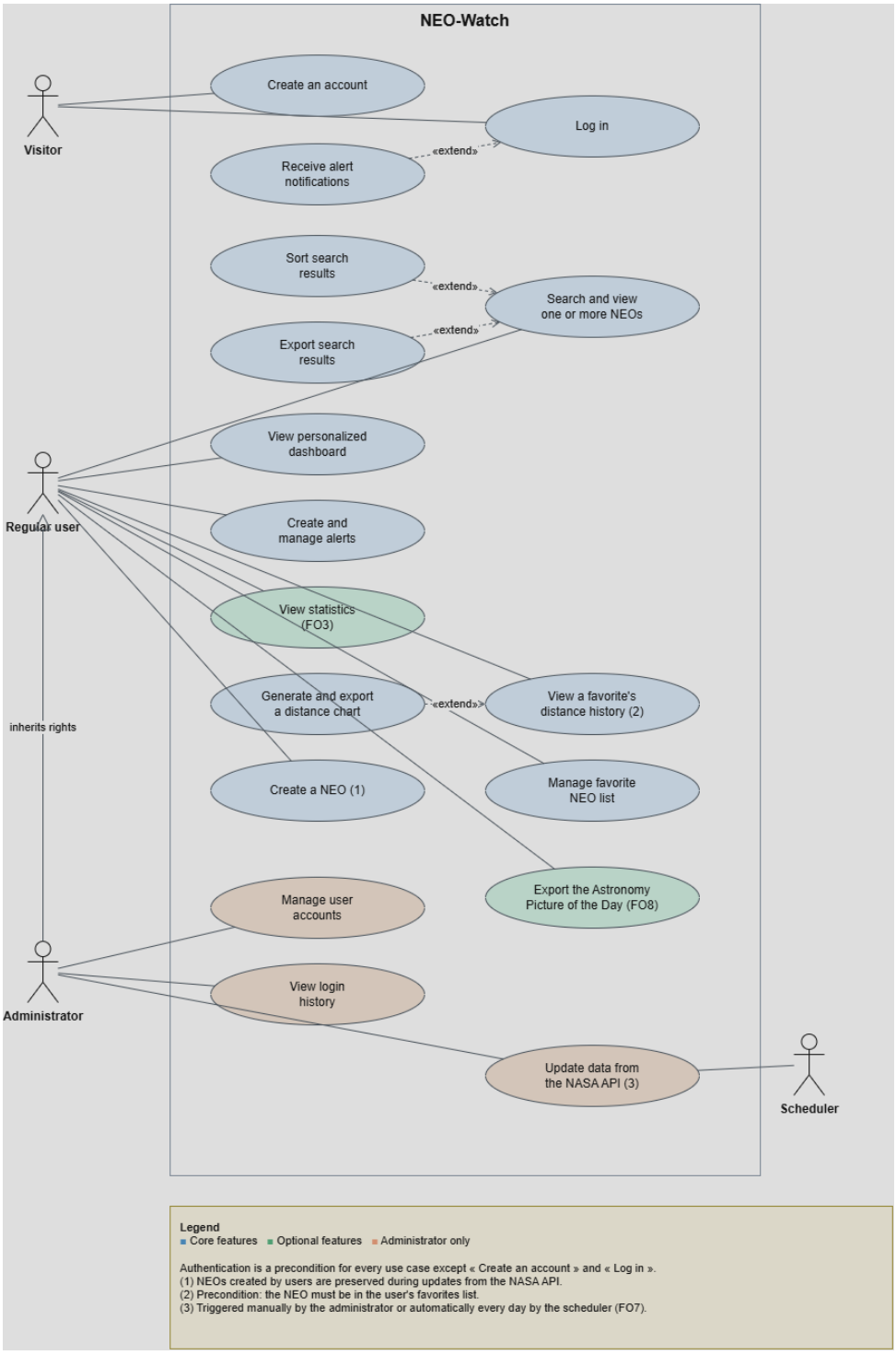


Figure 1: Use case diagram

The diagram shown in figure 1 distinguishes three actors.

The visitor only has access to account creation and login, since the application is secured. The regular user has access to the full set of consultation and personalization features: dashboard, search and export, creation of an observed NEO, management of their favorites and

the associated distance charts, definition of alerts and receipt of the corresponding notifications upon login. The administrator inherits these rights and adds account management, viewing the login history, and triggering the data update.

This inheritance relationship reflects the fact that an administrator is first and foremost a user, which avoids duplicating some fifteen use cases and is reflected in the model as a simple role attribute rather than a class hierarchy.

3.2 Layered architecture

The layered architecture is the foundation of our application. Each task required by the application is distributed between the different layers which have different roles.

The first one is the frontend. Its role is the only layer that users see and interact with. It shows everything that has to be displayed. In addition, it is also where people can press the buttons in order to navigate in the app. He will be able to choose the features he wants to try. Once the function is chosen, the request goes to the controller which is part of the backend.

The controller receives and sends https requests from the frontend, like the choice of users, or from the API. Its role is to link the API with the service, which is where all the process truly happens. In the classes Service, we have all the methods in order to make our application useful. It is like the brain of the project. For example, that is where the statistics are calculated, we can also create or delete a NEO. But, to be able to do that, it needs the data. Therefore, at this moment, the service calls the DAO (Data Access Object). The DAO receives the request in python of the service, converts it in sql, and sends that to the database. Once the response from the table is arrived, it is translated back to python, in an object that the app understands, and given to the concerned service.

3.3 Class diagram

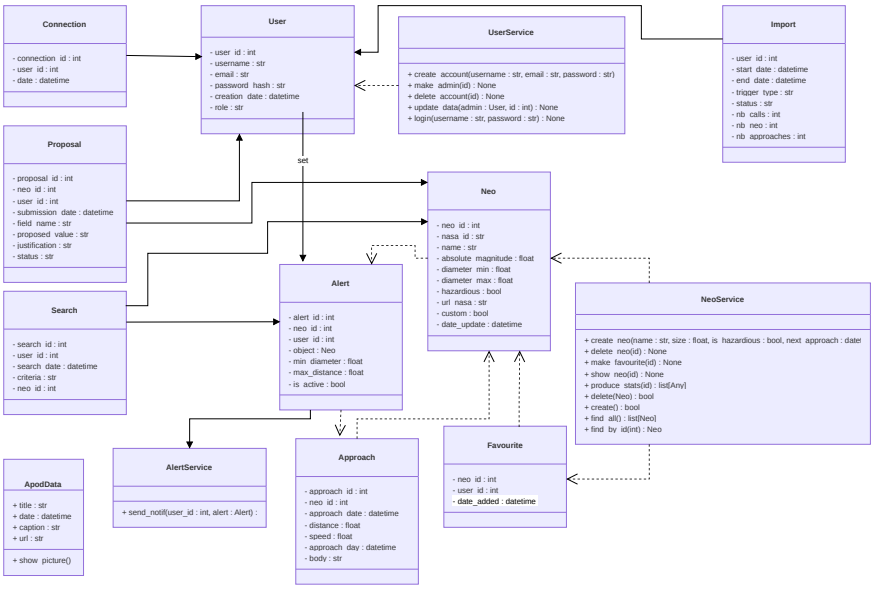


Figure 2: Class diagram

The class diagram contains most of the classes that we will need for the app. Many classes will be coded later, but we chose to show only these ones, because other classes about DAO or the controller for example, do not provide a lot of information as they are always present in this

kind of app. What we can see is the heart of the project, it is the structure of all the functions provided by the app.

Moreover, except for the classes Neo and User; the classes that have been created will be used only in one function. Even if we wanted to gather them as much as possible, in order to simplify our app, as all the functions are pretty different, we could not reduce their count.

Among all these, one class is isolated from the others. ApodData is the class that will fulfill the features of displaying the Picture of the Day. As its only role is to show this picture to the user, there is no link between an user information or a Neo, the class does not need any link.

3.4 Physical data model

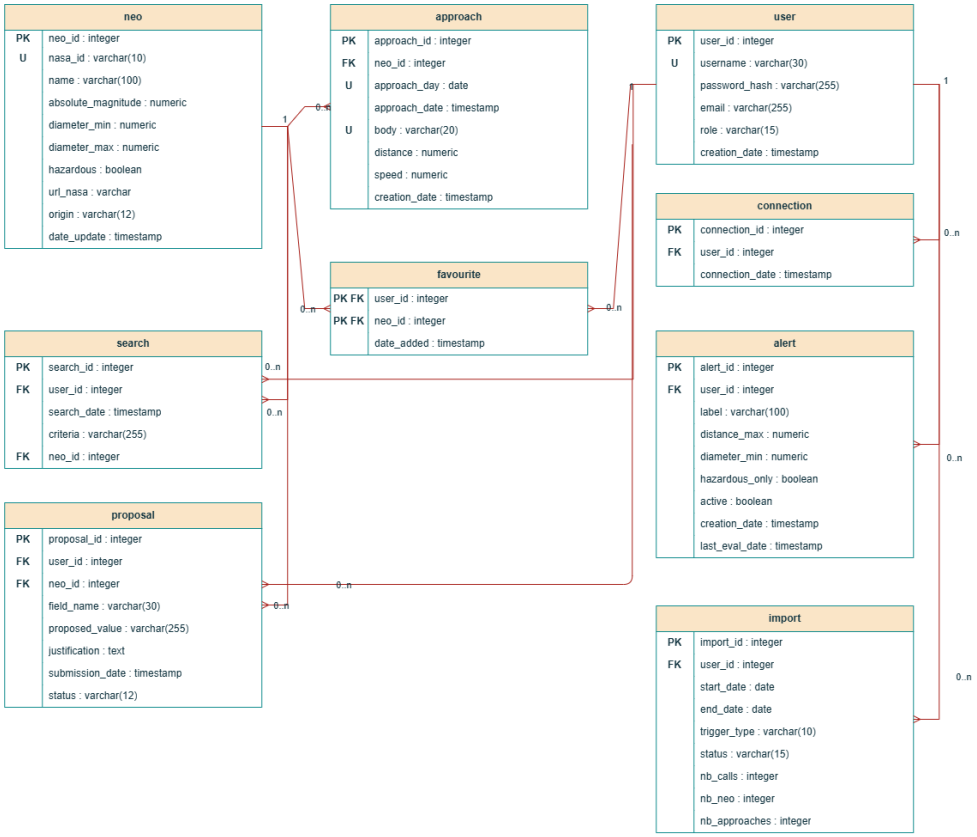


Figure 3: Physical data model

The model is built on nine tables, organized around two central entities. The `neo` table holds the stable characteristics of a near-Earth object, and the `approach` table its successive close passes near a body of the solar system. A NEO has zero to several close approaches, and each close approach concerns exactly one NEO.

Two tables rather than one. For each NEO, the API returns a list of dated close approaches, along with their distance and speed : the same object appears several times over a given period. A single table would have duplicated the physical characteristics at each of these approaches. Beyond the volume, the main risk concerns updates : diameter, magnitude and the hazard flag are regularly revised by NASA, and it would have been necessary to modify the twenty-nine rows of an object without error. We therefore split the data into two tables, `neo` and `approach`, linked by a foreign key. The decisive argument, however, is referential integrity: a favorite, an alert, or a proposed correction relates to a NEO and not to a close approach, and without a `neo` table these relationships would have no stable target to reference.

Primary keys and uniqueness constraints. The identifier provided by NASA guarantees uniqueness, but feature F4 allows a user to create a NEO they have observed themselves, and by definition such an object has no NASA identifier. We therefore use an auto-incremented technical key, and keep the NASA identifier in a separate column, nullable and subject to a uniqueness constraint. Imported objects and user-created objects thus coexist in the same table without either case having to be treated as an exception.

Identifying close approaches. The API provides no approach identifier. A close approach is therefore recognized by the triplet formed by the object, the day of the pass, and the body approached, on which a uniqueness constraint is placed. The day is used rather than the full timestamp because successive refinements of the orbital solutions shift the announced time : a key based on the timestamp would produce a duplicate at each update, whereas two passes of the same object toward the same body on the same day are physically impossible. The precise timestamp is still kept, for display purposes. All dates are expressed in Coordinated Universal Time as used by the API.

Preserving user-created objects. A refresh would overwrite user-entered objects on every import. To prevent this, we add an `origin` column to the `neo` table, which takes the value `NASA` or `USER`. The insert or update query executed during an import is restricted by a `WHERE origin = 'NASA'` clause, which makes user-created entries not affected by successive reloads.

The relationship between users and objects. The `favourite` table resolves the many-to-many relationship between users and objects, a user being able to follow several NEOs and a NEO being followed by several users. Its composite primary key inherently prevents the same object from being favorited twice by the same person. The date of addition belongs to the link itself rather than to either entity, and is therefore stored in the association table.

Storing statistics. The statistics expected in FO3 : number of hazardous objects over a period, minimum distance, averages, could be precomputed in a table refreshed at each import, which would speed up display. We chose the opposite solution: only raw data is stored, and all aggregates are computed in SQL at query time. The import window represents a few tens of thousands of approach rows, a volume for which aggregation on PostgreSQL is immediate, and this solution avoids maintaining a redundancy that would need to be invalidated at every update. The cost is a longer response time as the window widens, which we consider acceptable at the scale of this project.

Import traceability. Finally, the `import` table keeps a record of each data load. It allows the administrator to view its history, as required by F1, and allows the application to know the period actually covered by the database before answering a request for statistics over a period that may not have been imported.

3.4.1 Variable dictionary

Variable	Type	Description
neo — stable characteristics of a near-Earth object		
`neo_id`	integer	Technical primary key
`nasa_id`	varchar(10)	NASA identifier, empty if the object was created by a user
`name`	varchar(100)	Name or provisional designation of the object

Variable	Type	Description
`absolute_magnitude`	numeric	Intrinsic brightness of the object
`diameter_min`	numeric	Lower bound of the estimated diameter, in meters
`diameter_max`	numeric	Upper bound of the estimated diameter, in meters
`hazardous`	boolean	Object classified as potentially hazardous by NASA
`url_nasa`	varchar	Link to the official JPL page
`origin`	varchar(12)	`NASA` or `USER`
`date_update`	timestamp	Date of the row's last refresh
approach — close pass of an object near a body		
`approach_id`	integer	Technical primary key
`neo_id`	integer	Object concerned
`approach_day`	date	Day of the pass, in UTC
`approach_date`	timestamp	Precise timestamp of the pass
`body`	varchar(20)	Body approached, Earth within our scope
`distance`	numeric	Minimum distance reached, in kilometers
`speed`	numeric	Relative speed at the time of the pass
`creation_date`	timestamp	Date the row was inserted into the database
user — account for accessing the application		
`user_id`	integer	Technical primary key
`username`	varchar(30)	Login identifier, unique
`password_hash`	varchar(255)	Password hash
`email`	varchar(255)	Contact address
`role`	varchar(15)	`USER` or `ADMIN`
`creation_date`	timestamp	Registration date
favourite — association between a user and a followed object		
`user_id`	integer	User concerned
`neo_id`	integer	Object marked as favorite
`date_added`	timestamp	Date the favorite was added
connection — login history		
`connection_id`	integer	Technical primary key
`user_id`	integer	Account concerned
`connection_date`	timestamp	Date and time of login
alert — monitoring criteria defined by a user		
`alert_id`	integer	Technical primary key
`user_id`	integer	Owner of the alert
`label`	varchar(100)	Name given by the user
`distance_max`	numeric	Proximity threshold, empty if this criterion is not used
`diameter_min`	numeric	Size threshold, empty if this criterion is not used
`hazardous_only`	boolean	Restricts the alert to objects classified as hazardous

Variable	Type	Description
active	boolean	Allows the alert to be suspended without deleting it
creation_date	timestamp	Date the alert was defined
last_eval_date	timestamp	Last evaluation, avoids reporting the same event twice
import — record of loads from the NASA API		
import_id	integer	Technical primary key
user_id	integer	Triggering administrator, empty if the import is automatic
start_date	date	Start of the imported period
end_date	date	End of the imported period
trigger_type	varchar(10)	MANUAL or BATCH
status	varchar(15)	N ON_GOING, DONE, PARTIAL, or FAIL
nb_calls	integer	Number of calls made to the API
nb_neo	integer	Number of objects inserted or updated
nb_approaches	integer	Number of close approaches inserted or updated
search — search history, kept for thirty days		
search_id	integer	Technical primary key
user_id	integer	Author of the search
search_date	timestamp	Timestamp of the search
criteria	varchar(255)	Criteria entered, in readable text
neo_id	integer	Object viewed, empty if this is a multi-criteria search
proposal — correction suggested by a user		
proposal_id	integer	Technical primary key
user_id	integer	Author of the proposal
neo_id	integer	Object concerned
field_name	varchar(30)	Field the user wants corrected
proposed_value	varchar(255)	Suggested value
justification	text	Free-text explanation
submission_date	timestamp	Date the proposal was submitted
status	varchar(12)	WAITING, READ, or REJECTED

3.5 Packages diagram

The application is built upon a strict layered architecture to separate concerns and ensure maintainability. The FRONTEND, built with Streamlit, manages the user interface and captures user inputs. It communicates via HTTP requests with the CONTROLLER layer, handled by FastAPI. The controller routes these requests to the SERVICE LAYER, which acts as the "brain" of the project, executing the core business logic and interacting with the external NASA API when data updates are required. Finally, the DAO (Data Access Object) is responsible for data persistence. It translates Python commands into SQL queries to interact with the local PostgreSQL DATABASE.

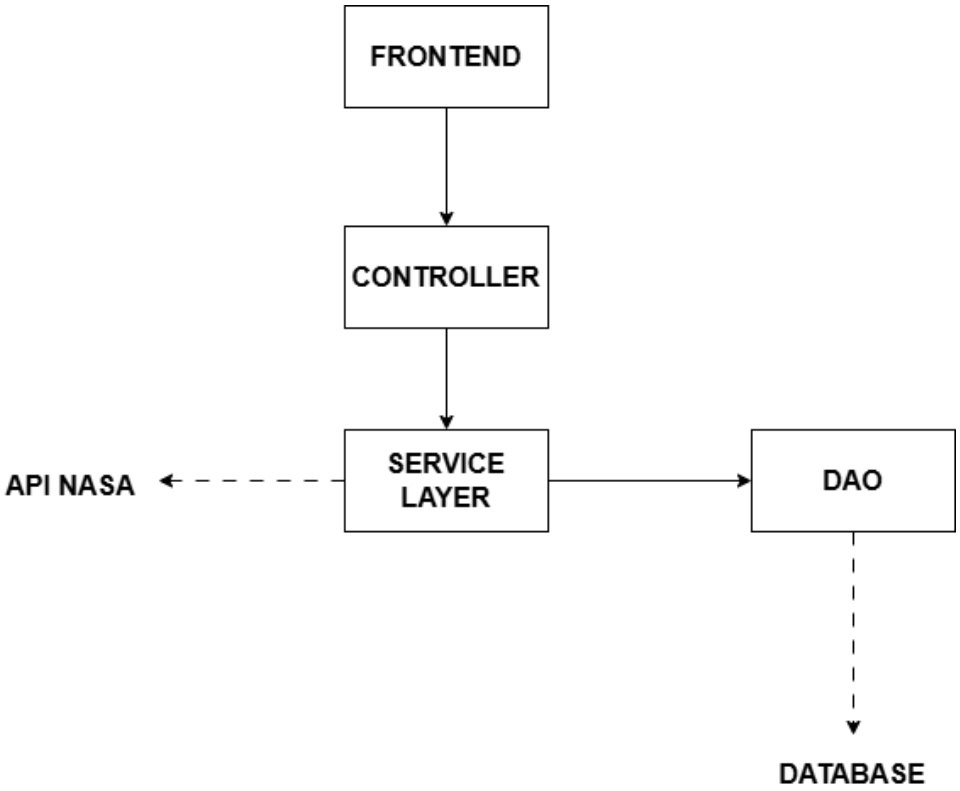


Figure 4: Packages diagram

3.6 Sequence diagram

This sequence diagram details the internal execution flow when a user searches for near-Earth objects based on specific criteria. When the search is submitted, the request travels from the Streamlit frontend down to the service layer. The DAO then executes a SQL JOIN query between the neo and approach tables in the local PostgreSQL database. This read operation relies entirely on our pre-populated database and does not trigger any external calls to the NASA API. The diagram highlights two alternative outcomes using an alt block: if matches are found, the data is formatted and passed back to the frontend to be displayed as a sortable table. An opt (optional) block illustrates the CSV export feature, which is handled purely on the client side without querying the database again. If no NEO matches the criteria, the system propagates an empty list back to the user interface to display a "No results found" message.

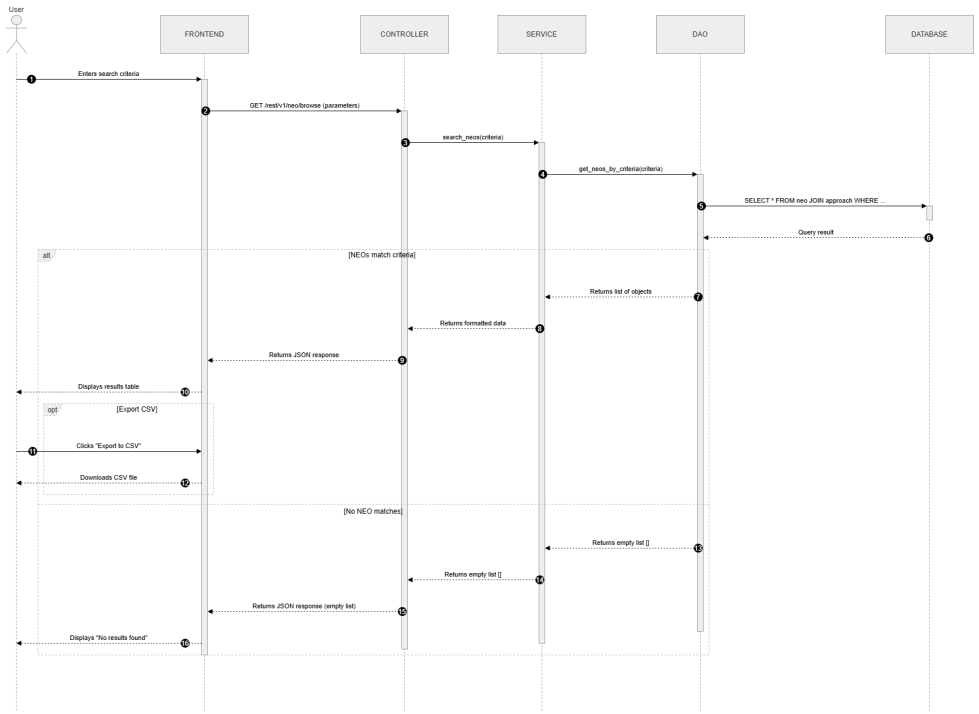


Figure 5: Sequence diagram

3.7 Endpoint table

3.8 Streamlit screens

NEO-Watch's frontend will be built with Streamlit. As no screens have been implemented yet, this section presents the planned screen structure, derived from the choices made in section 2.2.

Home / Dashboard. Entry point after login. Lists upcoming approaches within a selectable window, the count of potentially dangerous NEOs among them (flagged via the API's `is_potentially_hazardous_asteroid` field), and the closest NEO to Earth over a set period (F2), alongside the user's favorites and shortcuts to search, favorites and alerts.

Search. Lets the user query approaches over a date range, by name, id, dimensions or date (F3). Results are shown as a sortable table (distance in km and AU, speed in km/h) with a CSV export button, built directly from stored data with no additional call to the NASA API.

NEO detail page. Full record for one object: diameter range as returned by the API, distances, an "add to favorites" action, and for favorites, the distance history retrieved via `/neo/id` (F5), with CSV export of the history graph.

Create a NEO. Form for user-submitted observations (F4), limited to the fields an amateur observer can realistically provide; submitted objects are tagged `origin = USER` to protect them from the daily NASA import.

Alerts. Lets the user define and manage notification criteria (diameter, distance thresholds) (F6), and view past notifications received at login.

Statistics (FO3). Distributions over the objects currently in the database (size, share of hazardous objects), computed on demand in SQL rather than pre-aggregated.

Endpoint	Description
`POST /api/v1/auth/login`	Authenticate a user (regular or administrator) and initiate a session.
`GET /api/v1/dashboard`	Retrieve personalized dashboard metrics (upcoming approaches, hazardous NEOs count, closest NEO).
`GET /api/v1/neos`	Search the local database for NEOs and approaches based on query parameters (name, dates, dimensions).
`GET /api/v1/neos/{neo_id}`	Retrieve detailed characteristics and the complete close approach history for a specific NEO.
`POST /api/v1/neos`	Submit a new amateur observation to create a custom NEO in the database.
`GET /api/v1/users/me/favorites`	Get the authenticated user's list of favorite NEOs.
`POST /api/v1/users/me/favorites/{neo_id}`	Add a specific NEO to the user's favorites list.
`DELETE /api/v1/users/me/favorites/{neo_id}`	Remove a NEO from the user's favorites list.
`GET /api/v1/users/me/alerts`	Retrieve the user's active alert configurations and any pending notifications.
`POST /api/v1/users/me/alerts`	Create a new alert configuration (e.g., minimum diameter, maximum distance).
`GET /api/v1/stats`	Compute and retrieve global statistics (size distribution, hazardous proportion) from the local database.
`POST /api/v1/admin/import`	Trigger a manual data update pulling the latest data from the NASA API into the local database.

Table 2: Endpoints requiring user identification utilize the `/users/me/` path, ensuring secure and isolated access to personal favorites and alerts. The `/api/v1/admin/import` route is restricted to administrator accounts.

Picture of the Day (FO8). Displays the NASA APOD image and description.

Admin panel. Account management and connection history (F1), a manual "trigger update" button calling the same service as the scheduled daily job (FO7), and the status of the current/last import.

4 Components to implement and estimated durations

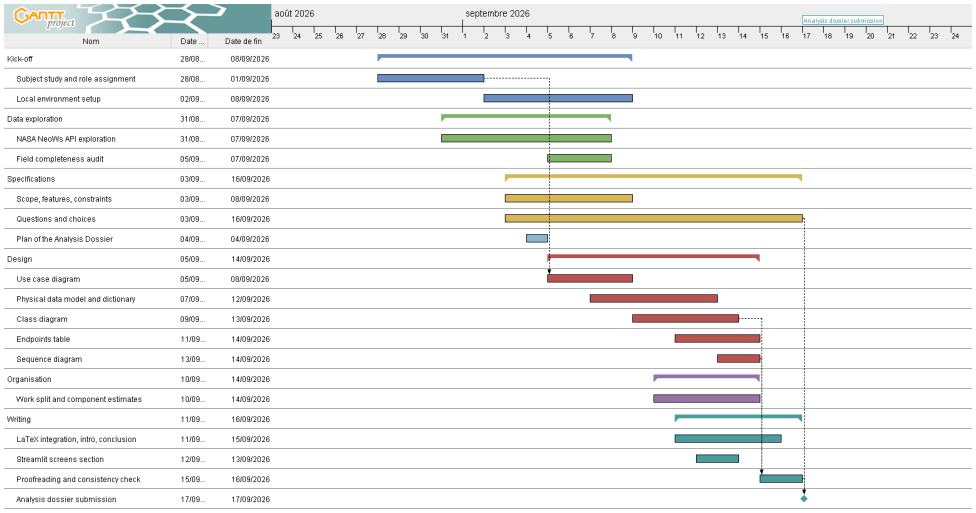


Figure 6: Gantt chart for phase 1 (17 September submission)

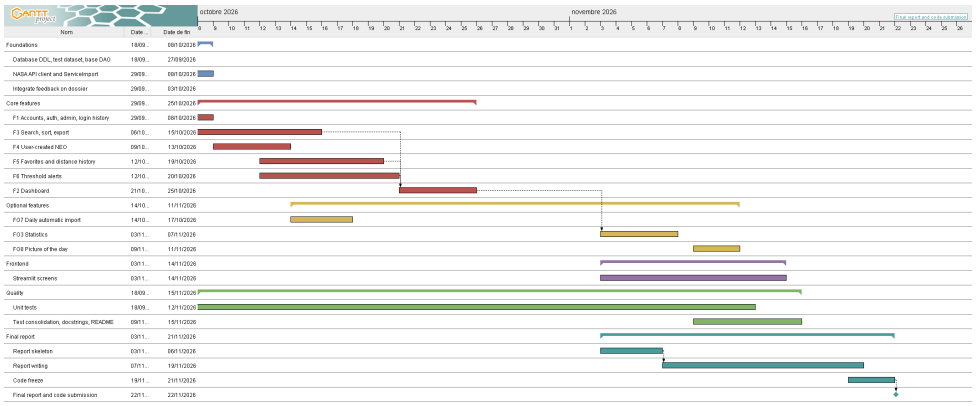


Figure 7: Gantt chart for phase 2 (final submission)

The two Gantt charts reflect the project's two official phases, separated by the submission of this analysis dossier on 17 September. Phase 1 lasted only three weeks, so tasks overlap heavily. Their order follows the dependencies between them: we explored the NASA NeoWs API before setting the scope, and the design deliverables were produced in sequence, from the physical data model to the class diagram, then the endpoints and the sequence diagram.

Phase 2 is shaped mainly by the academic calendar. The database is built first, because every other component depends on it, and its duration is extended to absorb revision for the 28 September exam. All required features (F1 to F6) are scheduled before the autumn break. The break is followed directly by the 2 November exam, so no new component starts during it, and it serves only as catch-up time. The optional features and the Streamlit frontend come after, since they rely on a backend API that is already stable and testable through Swagger. FO8 is

placed last because it has no dependencies and is the easiest to drop if we fall behind. Finally, the code freeze on 19 November leaves two days of review before the final submission.

5 Conclusion

At this level, our work has mainly focused on analysis and design rather than implementation. Following the group's own rule, we started by opening the NASA NeoWs API to understand its real constraints, endpoints, rate limits, and data structure, prior to making any technical commitment, since every other technical section depended on it.

This exploration shaped several key decisions. We chose /feed over /neo/browse as our import source, since our features query by date rather than by object, cutting the cost of populating the database from roughly 3,000 calls to about 150. We split neo and approach into two related tables to avoid duplicating an object's physical characteristics across its repeated approaches, and added an origin column so user-created NEOs (F4) survive the daily NASA import instead of being overwritten. We also fixed our shared vocabulary early, so naming stays consistent across the class diagram, the physical data model, and the endpoints, the kind of inconsistency that is clear to a reader.

These choices come with trade-offs we accept knowingly. Our two-year-past/one-year-future import window means NEO-Watch monitors close approaches rather than acting as a full catalogue of near-Earth objects, and any statistic tracking how an object's characteristics change over time is out of reach, not really a limitation of the window's size, but of the model itself.

The remaining work for the final report is the implementation of our features from F1 to F6 and also three chosen optional features (FO3, FO7, FO8), building on the layered architecture (view / services / DAO) defined during this analysis phase, followed by testing against the real API and completion of the final documentation.

Beyond these design trade-offs, NEO-Watch has a few limits by scope rather than by oversight. The app performs no orbital calculations or collision predictions of its own, it displays and organizes NASA's data but never actually reinterprets it, so its accuracy is entirely tied to the NASA API's own accuracy and uptime. Similarly, alerts and tracking are only as current as NASA's import cadence allows; NEO-Watch cannot notify users of an event faster than its source does. Finally, our own database can only ever complement NASA's, not replace or correct it: everything except user accounts, manually added NEOs, alerts, and favorites lists stays read-only by design. In other words, the main setback is that everything is hyper dependent on NASA's API.

More broadly, this project reflects a challenge that goes beyond NEO-Watch itself: as space-observation data becomes more abundant and more public, the real bottleneck shifts from access to usability. Tools like NEO-Watch are one small answer to that shift, turning a raw, technical API into something a non-specialist can actually use day to day. The same logic could extend well past near-Earth objects, to any domain where open scientific data outpaces the tools built to make sense of it.